

TP 5 : Traitement de flux avec Kafka Streams

Nettoyage de texte, analyse météo et comptage de clics

Module : Big Data – 2026

Enseignant : Mr. Abdelmajid BOUSSELHAM
ENSET Mohammedia

1. Objectif général du TP

L'objectif de ce TP est de développer des applications de traitement de flux avec **Kafka Streams**.

À travers trois exercices progressifs, vous allez apprendre à :

- lire des messages depuis des topics Kafka ;
- appliquer des transformations sur un flux de données ;
- filtrer des événements selon des règles métier ;
- router les messages vers différents topics ;
- réaliser des agrégations en temps réel ;
- intégrer Kafka Streams dans une architecture applicative basée sur Spring Boot.

Ce TP permet de comprendre comment Kafka Streams peut être utilisé dans des applications Big Data orientées temps réel.

2. Prérequis

Avant de commencer, vous devez disposer de :

- Java installé ;
- Apache Kafka installé ou exécuté avec Docker ;
- un broker Kafka fonctionnel ;
- Maven ou Gradle ;
- un IDE comme IntelliJ IDEA, Eclipse ou Visual Studio Code ;
- des connaissances de base en Java ;
- des notions de base sur Kafka : topic, producteur, consommateur, clé et valeur.

3. Rappels sur les concepts utilisés

3.1. Kafka Streams

Kafka Streams est une bibliothèque Java permettant de développer des applications de traitement de flux directement au-dessus d'Apache Kafka.

Elle permet notamment de :

- consommer des données depuis un ou plusieurs topics Kafka ;
- transformer les messages ;
- filtrer les événements ;
- regrouper les données ;
- effectuer des calculs en continu ;
- publier les résultats dans de nouveaux topics Kafka.

3.2. Concepts importants

Concept	Description
KStream	Représente un flux continu d'événements. Chaque message est traité comme un événement indépendant.
KTable	Représente une vue continuellement mise à jour. Elle est souvent obtenue après une agrégation.
KGrouperedStream	Représente un flux groupé par clé. Il est utilisé pour effectuer des agrégations comme <code>count</code> , <code>reduce</code> ou <code>aggregate</code> .
Topic d'entrée	Topic Kafka depuis lequel l'application lit les données.
Topic de sortie	Topic Kafka dans lequel l'application publie les résultats.
Dead Letter Topic	Topic utilisé pour stocker les messages invalides ou rejetés.

4. Exercice 1 : Nettoyage et validation de messages texte

4.1. Contexte

Dans cet exercice, vous allez développer une application Kafka Streams qui traite des messages texte envoyés dans un topic Kafka.

Chaque message reçu est une simple chaîne de caractères. L'application doit nettoyer le message, vérifier sa validité, puis l'envoyer vers le topic approprié.

4.2. Topics à utiliser

Vous devez créer les topics Kafka suivants :

Topic	Rôle
<code>text-input</code>	Topic contenant les messages texte bruts.
<code>text-clean</code>	Topic contenant les messages valides après nettoyage.
<code>text-dead-letter</code>	Topic contenant les messages invalides ou rejetés.

4.3. Création des topics

Créer les trois topics avec les commandes suivantes :

```
kafka-topics.sh --create \
  --topic text-input \
  --bootstrap-server localhost:9092 \
```

```
--partitions 1 \
--replication-factor 1

kafka-topics.sh --create \
--topic text-clean \
--bootstrap-server localhost:9092 \
--partitions 1 \
--replication-factor 1

kafka-topics.sh --create \
--topic text-dead-letter \
--bootstrap-server localhost:9092 \
--partitions 1 \
--replication-factor 1
```

4.4. Travail demandé

Développer une application Kafka Streams qui réalise les traitements suivants :

1. Lire les messages depuis le topic `text-input`.
2. Nettoyer chaque message en appliquant les opérations suivantes :
 - supprimer les espaces au début et à la fin ;
 - remplacer les espaces multiples par un seul espace ;
 - convertir le message en majuscules.
3. Vérifier si le message est valide ou non.
4. Envoyer les messages valides dans le topic `text-clean`.
5. Envoyer les messages invalides dans le topic `text-dead-letter`.

4.5. Règles de validation

Un message doit être considéré comme invalide dans les cas suivants :

- le message est vide ;
- le message contient uniquement des espaces ;
- le message contient un mot interdit ;
- le message dépasse 100 caractères.

Les mots interdits à utiliser sont :

```
HACK
SPAM
XXX
```

4.6. Exemples de tests

Message envoyé	Résultat attendu	Topic cible
Bonjour Kafka Streams	BONJOUR KAFKA STREAMS	text-clean
hello world	HELLO WORLD	text-clean
	Message rejeté	text-dead-letter
this is spam message	Message rejeté	text-dead-letter
message contenant hack	Message rejeté	text-dead-letter

Remarque : dans le tableau précédent, le symbole ~ représente un espace.

4.7. Test de l'application

Pour envoyer des messages dans le topic d'entrée :

```
kafka-console-producer.sh \  
--topic text-input \  
--bootstrap-server localhost:9092
```

Pour vérifier les messages valides :

```
kafka-console-consumer.sh \  
--topic text-clean \  
--from-beginning \  
--bootstrap-server localhost:9092
```

Pour vérifier les messages invalides :

```
kafka-console-consumer.sh \  
--topic text-dead-letter \  
--from-beginning \  
--bootstrap-server localhost:9092
```

4.8. Questions

1. Quel est le rôle du topic `text-dead-letter` ?
2. Pourquoi est-il important de nettoyer les messages avant de les traiter ?
3. Pourquoi faut-il convertir le texte en majuscules avant de vérifier les mots interdits ?
4. Comment peut-on améliorer cette application pour gérer une liste de mots interdits stockée dans un fichier ou une base de données ?

5. Exercice 2 : Analyse de données météorologiques

5.1. Contexte

Une entreprise collecte des données météorologiques en temps réel à partir de plusieurs stations.

Chaque station envoie ses mesures dans un topic Kafka nommé `weather-data`. Vous devez développer une application Kafka Streams permettant de filtrer, transformer et agréger ces mesures.

5.2. Format des messages

Chaque message du topic `weather-data` respecte le format suivant :

```
station,temperature,humidity
```

Exemple :

```
Station1,25.3,60  
Station2,35.0,50  
Station2,40.0,45  
Station1,32.0,70
```

5.3. Description des champs

Champ	Description
station	Identifiant de la station météorologique. Exemple : Station1 .
temperature	Température mesurée en degrés Celsius. Exemple : 25.3.
humidity	Taux d'humidité en pourcentage. Exemple : 60.

5.4. Topics à utiliser

Créer les topics Kafka suivants :

Topic	Rôle
weather-data	Topic contenant les mesures météorologiques brutes.
station-averages	Topic contenant les moyennes calculées par station.

5.5. Création des topics

```
kafka-topics.sh --create \
  --topic weather-data \
  --bootstrap-server localhost:9092 \
  --partitions 1 \
  --replication-factor 1

kafka-topics.sh --create \
  --topic station-averages \
  --bootstrap-server localhost:9092 \
  --partitions 1 \
  --replication-factor 1
```

5.6. Travail demandé

Développer une application Kafka Streams qui réalise les traitements suivants :

1. Lire les messages depuis le topic **weather-data** sous forme de **KStream**.
2. Extraire les champs **station**, **temperature** et **humidity**.
3. Conserver uniquement les relevés dont la température est supérieure à 30°C.
4. Convertir la température de Celsius vers Fahrenheit.
5. Regrouper les données par station.
6. Calculer pour chaque station :
 - la température moyenne ;
 - le taux d'humidité moyen.
7. Publier les résultats dans le topic **station-averages**.

5.7. Formule de conversion

La conversion de Celsius vers Fahrenheit se fait avec la formule suivante :

$$F = \left(C \times \frac{9}{5} \right) + 32$$

Avec :

- C : température en degrés Celsius ;
- F : température en degrés Fahrenheit.

5.8. Exemple de traitement

Données envoyées dans le topic `weather-data` :

```
Station1,25.3,60
Station2,35.0,50
Station2,40.0,45
Station1,32.0,70
```

Après filtrage, les données conservées sont :

```
Station2,35.0,50
Station2,40.0,45
Station1,32.0,70
```

Après conversion en Fahrenheit :

```
Station2,95.0,50
Station2,104.0,45
Station1,89.6,70
```

Résultat attendu dans le topic `station-averages` :

```
Station2 : Temperature moyenne = 99.5 F, Humidite moyenne = 47.5 %
Station1 : Temperature moyenne = 89.6 F, Humidite moyenne = 70.0 %
```

5.9. Contraintes techniques

Votre application doit respecter les contraintes suivantes :

- utiliser les concepts `KStream`, `KGroupedStream` et `KTable` ;
- utiliser une sérialisation correcte des clés et des valeurs ;
- gérer les messages mal formés ;
- éviter l'arrêt brutal de l'application en cas d'erreur dans une ligne ;
- ajouter un hook d'arrêt propre de l'application.

5.10. Questions

1. Pourquoi doit-on regrouper les données par station avant de calculer les moyennes ?
2. Quelle est la différence entre `KStream` et `KTable` ?
3. Pourquoi le résultat d'une agrégation est-il souvent représenté sous forme de `KTable` ?
4. Comment gérer un message mal formé comme `Station1,error,60` ?
5. Pourquoi Kafka Streams est adapté à ce type de traitement ?

6. Exercice 3 : Comptage de clics avec Kafka Streams et Spring Boot

6.1. Contexte

Dans cet exercice, vous allez concevoir une petite architecture orientée événements permettant de compter les clics des utilisateurs en temps réel.

Une application web Spring Boot affiche un bouton. Chaque clic sur ce bouton envoie un événement dans Kafka. Une application Kafka Streams consomme ces événements, calcule le nombre de clics, puis publie le résultat dans un autre topic. Une API REST permet ensuite de consulter le nombre de clics.

6.2. Architecture demandée

L'application complète doit être composée de trois parties :

1. **Producteur Web** : une application Spring Boot avec une interface contenant un bouton.
2. **Traitement Kafka Streams** : une application qui calcule le nombre de clics.
3. **Consommateur REST** : une application Spring Boot qui expose les résultats via une API REST.

6.3. Topics à utiliser

Créer les topics Kafka suivants :

Topic	Rôle
clicks	Topic contenant les événements de clics générés par l'application web.
click-counts	Topic contenant les résultats du comptage des clics.

6.4. Création des topics

```
kafka-topics.sh --create \  
  --topic clicks \  
  --bootstrap-server localhost:9092 \  
  --partitions 1 \  
  --replication-factor 1  
  
kafka-topics.sh --create \  
  --topic click-counts \  
  --bootstrap-server localhost:9092 \  
  --partitions 1 \  
  --replication-factor 1
```

6.5. Partie 1 : Producteur Web

Développer une application Spring Boot qui expose une page web simple contenant un bouton :

Cliquez ici

À chaque clic sur ce bouton, l'application doit envoyer un message vers le topic Kafka `clicks`.

Le message envoyé doit contenir :

- une clé représentant l'utilisateur, par exemple `userId`;
- une valeur représentant l'action, par exemple `click`.

Exemple :

```
key = user1
value = click
```

6.6. Partie 2 : Application Kafka Streams

Développer une application Kafka Streams qui :

1. consomme les événements depuis le topic `clicks`;
2. calcule le nombre de clics;
3. permet soit un comptage global, soit un comptage par utilisateur;
4. publie les résultats dans le topic `click-counts`.

Deux variantes sont possibles :

- **Variante 1** : calculer le nombre total de clics ;
- **Variante 2** : calculer le nombre de clics par utilisateur.

6.7. Partie 3 : Consommateur REST

Développer une application Spring Boot qui consomme les résultats depuis le topic `click-counts`.

Cette application doit exposer une API REST permettant de consulter le nombre de clics.

Endpoint demandé :

```
GET /clicks/count
```

Exemple de réponse pour un comptage global :

```
{
  "totalClicks": 125
}
```

Exemple de réponse pour un comptage par utilisateur :

```
{
  "user1": 45,
  "user2": 30,
  "user3": 50
}
```


6.8. Travail demandé

Vous devez réaliser les tâches suivantes :

1. Créer les topics Kafka `clicks` et `click-counts`.
2. Développer l'application Spring Boot productrice de clics.
3. Créer une interface web simple contenant un bouton.
4. Envoyer un message Kafka à chaque clic.
5. Développer l'application Kafka Streams pour compter les clics.
6. Publier les résultats dans le topic `click-counts`.
7. Développer une API REST permettant de consulter les résultats.
8. Tester le scénario complet.

6.9. Scénario de test

Pour valider votre travail, suivre les étapes suivantes :

1. Démarrer Kafka.
2. Créer les topics nécessaires.
3. Lancer l'application productrice web.
4. Lancer l'application Kafka Streams.
5. Lancer l'application REST.
6. Cliquer plusieurs fois sur le bouton.
7. Consulter l'endpoint REST.

Exemple de test avec `curl` :

```
curl http://localhost:8080/clicks/count
```

6.10. Résultat attendu

Après plusieurs clics, l'API REST doit retourner un nombre qui évolue progressivement.
Exemple :

```
{  
  "totalClicks": 10  
}
```

Après cinq nouveaux clics, le résultat peut devenir :

```
{  
  "totalClicks": 15  
}
```

7. Livrables attendus

À la fin du TP, vous devez remettre :

- le code source des applications développées ;
- les commandes utilisées pour créer les topics Kafka ;

- des captures d’écran montrant :
 - les topics créés ;
 - les messages envoyés ;
 - les messages consommés ;
 - les résultats obtenus ;
 - l’API REST fonctionnelle pour l’exercice 3.
- une courte description de l’architecture réalisée ;
- une explication des principales difficultés rencontrées et des solutions proposées.